Qiskit Fall Fest 2026 — Phase 1 Master Guide

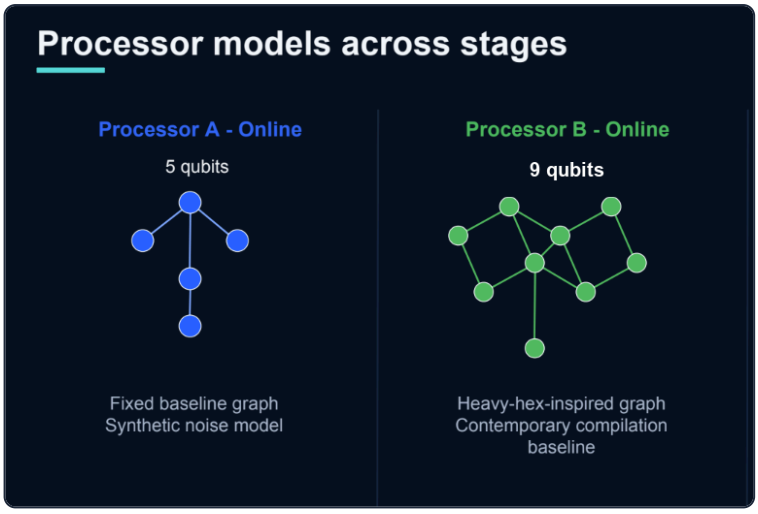
Official Unified Challenge Kit & Operational Reference | SRM University-AP

CRITICAL OPERATIONAL UPDATES & SUBMISSION DETAILS

- **Extended Deadline:** Wednesday, **October 14, 2026, at 23:59 IST (Midnight IST)**.
- **Submission Portal:** Submit your public GitHub repository URL at **https://qffsrmap2026.com/learning/hackathon**.
- **Official Drive Kit: https://drive.google.com/drive/folders/1caxlXXeqjOkLmXGlimy_Etw0uPHSwyga**.
- **Phase 1 Scope:** Strictly **Processor A & Processor B**. Processor C and custom Processor D are exclusively reserved for Phase 2 offline finalists. You do **not** need to implement or benchmark C or D for Phase 1!
- **Processor Modeling Challenge:** Pre-compiled `.json` files are intentionally **not provided**. Deducing, specifying, and creatively designing the coupling graphs in Qiskit is Challenge Step 1.
- **Self-Contained Data:** No external dataset downloads are required. All tasks use programmatic generation, standard libraries (Qiskit Nature, NetworkX, scikit-learn), or inline parameters.

1. Universal Challenge Framework: Algorithm ↔ Architecture Co-Design

Quantum algorithms are conceived logically, but real quantum processors have physical constraints: finite connectivity, limited qubit counts, and noisy operations. The core objective of Phase 1 is to demonstrate that you can implement an end-to-end quantum solution, map and benchmark it across two distinct physical architectures (**Processor A** and **Processor B**), and systematically evaluate how physical connectivity, gate routing, depth, and noise affect execution fidelity.



IMPORTANT NOTICE: COUPLING MAP CREATIVE FREEDOM & REFERENCE ARCHITECTURES

The coupling maps shown in the Processor A/B Benchmark Configuration should be treated as **reference architectures only**, not as mandatory coupling maps.

For Phase 1, participants are expected to build their Processor A and Processor B models from scratch based on the specified qubit counts, gate requirements, and benchmark constraints. During this design process, your coupling map may differ from another team's coupling map—**this is intentional and completely acceptable**.

The goal is to explore how different connectivity choices affect routing, SWAP count, circuit depth, CX count, and overall performance.

There is no single "correct" coupling map. Your architecture is part of your solution.

Parameter	Processor A (Baseline Reference)	Processor B (Contemporary Reference)
Physical Qubit Budget	**5 Qubits** (Nodes 0 to 4)	**9 Qubits** (Nodes 0 to 8)
Coupling Topology *(Reference Only)*	**Reference baseline:** Linear chain `[(0,1),(1,2),(2,3),(3,4)]` or branching tree `[(0,1),(1,2),(1,3),(3,4)]`. *You are free to design and justify any valid 5-qubit graph.*	**Reference baseline:** Sparse 9-qubit heavy-hex-inspired graph (12 edges): `(0,1), (1,2), (1,6), (2,3), (2,6), (2,5), (3,4), (4,5), (5,6), (6,7), (6,8), (8,0)`. *You are free to design and justify any valid 9-qubit graph.*
Basis Gate Set	`{rz, sx, x, cx}`	`{rz, sx, x, cx}`
Synthetic Noise Model *(For noisy evaluation)*	• 1-qubit depolarizing: 0.1% (p = 0.001) • 2-qubit CX depolarizing: 0.5% (p = 0.005) • Readout error: 1.0% (p = 0.01)	Apply identical synthetic noise rates to Processor B so performance differences isolate connectivity effects.
Transpilation Baseline	Optimization Level = 1, Seed = 42, Shots = 4096 (unless PS specifies otherwise)	Optimization Level = 1, Seed = 42, Shots = 4096

2. Participant Workflow & Authoritative Repository Structure

Each team (2 to 6 members) selects **exactly one** problem statement from the 20 available choices. Your repository must make your work transparent, modular, and reproducible. Submit a single public GitHub repository at **https://qffsrmap2026.com/learning/hackathon** structured exactly as follows:

```
project/
├── README.md                        # Problem statement selected, setup, methodology, key results
├── main.ipynb                       # End-to-end reproducible experiment notebook
├── requirements.txt                 # Software dependencies (qiskit, qiskit-aer, networkx, etc.)
├── data/
│   ├── raw/                         # Raw data or generated inputs
│   └── processed/                   # Cleaned arrays or matrices
├── processors/
│   ├── processor_A.json             # Participant-defined Processor A coupling map & config
│   └── processor_B.json             # Participant-defined Processor B coupling map & config
├── src/
│   ├── __init__.py
│   ├── problem.py                   # Problem formulation & cost Hamiltonian construction
│   ├── processors.py                # Topology graph definition & noise model loaders
│   ├── routing.py                   # Transpilation & layout routing routines
│   ├── metrics.py                   # Circuit metrics (depth, CX, SWAP, fidelity)
│   └── visualization.py             # Plotting & coupling graph visualizers
├── results/
│   ├── tables/                      # Exported CSV/JSON benchmark comparison tables
│   └── figures/                     # Exported comparison figures (depth, energy curves, SWAP overhead)
├── report.md                        # Architectural analysis, trade-off interpretation, limitations
└── assets/
    └── qff_phase1_latex_images/          # Optional LaTeX equation renders or visual assets
        ├── eq_qubit_state.png
        ├── eq_bell_state.png
        ├── eq_graph.png
        ├── eq_bell_fidelity.png
        ├── eq_zz_expectation.png
        └── eq_standard_error.png
```

3. Detailed Problem Statement Directory (20 Choices Across 5 Tracks)

Below are all 20 challenge choices. Every problem statement is designed to be 100% self-contained—no external downloads required.

Track 1: Quantum Chemistry (VQE / Molecular Systems)

PS-C1: Molecular Ground-State Energy (H_2)

Task: Compute the ground-state energy of H_2 at equilibrium bond distance (R = 0.74 Å) using VQE with a hardware-efficient or UCCSD ansatz. Validate against classical full configuration interaction (-1.137 Hartree). Benchmark transpiled depth, CX count, and energy accuracy on Processor A vs B.

Data / Setup: Construct Hamiltonian via Qiskit Nature: `Molecule(geometry=[['H', [0., 0., 0.]], ['H', [0., 0., 0.74]]], charge=0, multiplicity=1)` or use the standard 4-qubit Jordan-Wigner Hamiltonian.

PS-C2: Diatomic Potential Energy Surface (H_2 / LiH)

Task: Map the potential energy dissociation curve across 5–7 bond distances (e.g. 0.5 to 2.5 Å). Keep the variational ansatz fixed and evaluate how transpiled gate counts and optimizer convergence differ between Processor A and B as molecular geometry stretches.
Data / Setup: Programmatically sweep bond distances in Python using Qiskit Nature or precomputed electronic integrals.

PS-C3: Hardware-Efficient Triatomic System (H_2O / H_3^+)

Task: Target a reduced model for a triatomic molecule using active space reduction or qubit tapering to fit within 4–8 logical qubits. Compare ansatz scalability and SWAP routing overhead on Processor A vs B.
Data / Setup: Generate reduced Hamiltonian via Qiskit Nature active space transformer or freeze-core approximation.

PS-C-OI: Open Innovation — Custom Quantum Chemistry

Task: Propose and solve a custom chemistry problem (e.g., excited state calculation, dipole moment, or custom transition metal bond) within 4–8 qubits on Processor A vs B.
Data / Setup: Define Hamiltonian construction and classical benchmark reference in your repository.

Track 2: Quantum Optimization (QAOA / QUBO)

PS-O1: Max-Cut on Weighted Graphs

Task: Solve Max-Cut on a 6-node weighted graph using QAOA (p=1 or p=2). Compare cut value, approximation ratio, circuit depth, and SWAP routing overhead on Processor A vs Processor B.
Data / Setup: Generate a 6-node random graph programmatically using NetworkX: `nx.erdos_renyi_graph(n=6, p=0.5, seed=42)` with integer edge weights 1–5.

PS-O2: Portfolio Optimization Under Budget Constraints

Task: Formulate a 4-asset portfolio selection problem with return maximization and risk penalty: minimize $w^T\Sigma w - q\mu^T w$ subject to $\sum w_i = k$. Encode as QUBO and solve via QAOA or VQE on A vs B.
Data / Setup: Use inline returns μ = [0.12, 0.18, 0.10, 0.22], budget k = 2, and risk covariance matrix Σ = [[0.04, 0.01, 0.02, 0.01], [0.01, 0.09, 0.01, 0.03], [0.02, 0.01, 0.05, 0.02], [0.01, 0.03, 0.02, 0.08]].

PS-O3: 0/1 Knapsack Problem

Task: Formulate 0/1 Knapsack as a QUBO with quadratic penalty for weight overflow. Solve via QAOA and benchmark against classical brute force. Compare routing cost on A vs B.
Data / Setup: Use inline values = [10, 4, 7, 5, 8], weights = [5, 3, 4, 2, 6], and maximum capacity W = 15.

PS-O-OI: Open Innovation — Custom Quantum Optimization

Task: Formulate and solve a custom QUBO problem (e.g., TSP, graph coloring, facility location) using QAOA/VQE on Processor A vs B.
Data / Setup: Specify problem definition, classical verification, and objective formulation clearly in README.

Track 3: Quantum Simulation (Hamiltonian Dynamics)

PS-S1: Transverse-Field Ising Model (TFIM) Ground State

Task: Estimate the ground state energy and magnetization of a 4-spin 1D TFIM chain ($H = -J \sum Z_i Z_{i+1} - g \sum X_i$) across external fields $g \in \{0.5, 1.0, 1.5\}$ using VQE on Processor A vs B.
Data / Setup: Construct Hamiltonian directly via Qiskit `SparsePauliOp`: terms 'ZZII', 'IZZI', 'IIZZ' (J=1.0) and 'XIII', 'IXII', 'IIXI', 'IIIX' (coupling g).

PS-S2: Real-Time Trotterized Spin Dynamics

Task: Simulate real-time unitary evolution $U(t) = \exp(-iHt)$ of the 4-spin TFIM under first-order Trotter decomposition up to $t = 2.0$. Measure $\langle Z_0(t) \rangle$ and compare accuracy and SWAP overhead on A vs B.
Data / Setup: Initial state $|0000\rangle$ or Néel state $|0101\rangle$. Compute exact matrix exponential classically via `scipy.linalg.expm` for comparison.

PS-S3: Noise Accumulation in Trotter Circuits

Task: Study the degradation of Trotterized simulation accuracy as a function of Trotter steps (n=1, 2, 4) under the official synthetic noise model on Processor A vs Processor B.
Data / Setup: Same 4-spin TFIM setup. Simulate using Qiskit `AerSimulator` with the specified depolarizing error model.

PS-S-OI: Open Innovation — Custom Quantum Simulation

Task: Propose a custom spin chain, fermionic model (Fermi-Hubbard), or time-dependent Hamiltonian simulation on A vs B.
Data / Setup: Provide exact numerical diagonalization baseline in Python.

Track 4: Quantum Machine Learning (QML)

PS-Q1: Variational Quantum Classifier (VQC)

Task: Build and train a 4-qubit Variational Quantum Classifier with angle encoding and parameterized rotation layers. Compare classification accuracy, training convergence, and compiled circuit depth on Processor A vs B.
Data / Setup: Generate 200 synthetic data samples (4 features, binary class) using `sklearn.datasets.make_classification(n_samples=200, n_features=4, n_informative=3, random_state=42)` with an 80/20 train/test split.

PS-Q2: Quantum Kernel Estimator & Support Vector Machine (QSVM)

Task: Construct a quantum kernel matrix using fidelity state overlap $|\langle \varphi(x) | \varphi(x') \rangle|^2$ on a 4-qubit register. Train a classical SVM with the quantum kernel. Compare kernel evaluation cost on A vs B.
Data / Setup: Generate dataset using `sklearn.datasets.make_moons(n_samples=100, noise=0.1, random_state=42)`.

PS-Q3: Noise-Resilient Classification & Error Mitigation

Task: Evaluate a trained VQC under ideal vs synthetic device noise. Apply at least one error mitigation strategy (e.g., Zero-Noise Extrapolation via Mitiq or readout measurement mitigation) on Processor A vs B.
Data / Setup: Use the same synthetic classification dataset and official synthetic noise parameters.

PS-Q-OI: Open Innovation — Custom QML

Task: Design a custom QML model (e.g. quantum autoencoder, generative QGAN, or hybrid neural net) on Processor A vs B.
Data / Setup: Clearly document feature encoding and classical comparison baseline in your repository.

Track 5: Post-Quantum Cryptography (Quantum-Safe Systems)

PS-P1: Quantum-Safe Hybrid Key Exchange

Task: Implement and benchmark a hybrid key encapsulation mechanism combining classical ECDH (P-256) with NIST PQC standard ML-KEM-768 (Kyber). Measure handshake latency, ciphertext size, and memory overhead on simulated standard (Platform A) vs constrained (Platform B) CPU environments.
Data / Setup: Use public Python libraries (`pyca/cryptography` and `oqs-python` / liboqs or pure-Python ML-KEM reference).

PS-P2: Cryptographic Inventory & Migration Scanner

Task: Develop a static analysis scanner (AST or pattern-based) that inspects a codebase to detect quantum-vulnerable cryptography (RSA, DSA, DH, ECC) and flags migration paths to NIST FIPS 203/204 standards.
Data / Setup: Run the scanner against any public open-source Git repository (e.g. Requests, Flask) and report precision, recall, and scan execution time.

PS-P3: PQC Algorithm Benchmark Across Parameter Sets

Task: Perform a reproducible empirical benchmark of ML-KEM (512, 768, 1024) and ML-DSA (44, 65, 87) measuring key generation time, encapsulation/signing time, and public key / signature sizes across 100 iterations.
Data / Setup: No external dataset required. Use standardized NIST FIPS parameters and generate boxplots/tables.

PS-P-OI: Open Innovation — Custom PQC Systems

Task: Propose a custom quantum-safe systems challenge (e.g. PQC in TLS 1.3, stateful hash signatures XMSS/LMS, or side-channel resistance analysis).
Data / Setup: Define threat model and benchmarking setup in repository.

4. Judging Rubric & Phase 1 Participant Checklist

Evaluation Criteria	Weight	What Judges Will Look For
Scientific & Algorithmic Correctness	**30 pts**	Does the algorithm execute without errors? Are results compared against an exact or classical theoretical reference?
Processor A vs B Architectural Analysis & Creativity	**35 pts**	Quantitative comparison: circuit depth, CX count, SWAP routing overhead, and noise impact. Evaluation of how creatively the participant designed and reasoned about their coupling topologies.
Reproducibility & Code Quality	**20 pts**	Clean environment, pinned requirements.txt, structured repository, and end-to-end runnable notebook.
Documentation & Visualizations	**15 pts**	Clear README, coupling map diagrams, and legible performance comparison plots.

PARTICIPANT CLOSING CHECKLIST (BEFORE OCT 14, 23:59 IST)

- [ ] Exactly **ONE** Problem Statement selected and documented in README.md.
- [ ] Core solution implemented and verified against classical / exact reference.
- [ ] Coupling maps for Processor A (5q) and Processor B (9q) creatively defined, justified, and transpiled.
- [ ] Quantitative comparison table generated (Depth, CX Count, SWAP overhead, Accuracy on A vs B).
- [ ] Repository follows the required file structure (including `main.ipynb`, `requirements.txt`, `processors/`, `results/`).
- [ ] Public GitHub repository submitted at **https://qffsrmap2026.com/learning/hackathon**.
- *Note: Processor C and Processor D are NOT part of Phase 1 and are not required.*